

SISTEMAS OPERATIVOS

3004610 - 1

Germán Sánchez Torres, I.S., M.Sc., Ph.D.

Profesor, Facultad de Ingeniería · Programa de Ingeniería de Sistemas

Universidad del Magdalena, Santa Marta

Teléfono: +57 605 421 4079 ext. 1138 · 301 683 6593

Edificio Docente, cubículo 3D401

Correo: gsanchez@unimagdalena.edu.co · sanchez.gt@gmail.com



Memoria compartida
(System V IPC)
shmget, shmat, shmdt, shmctl

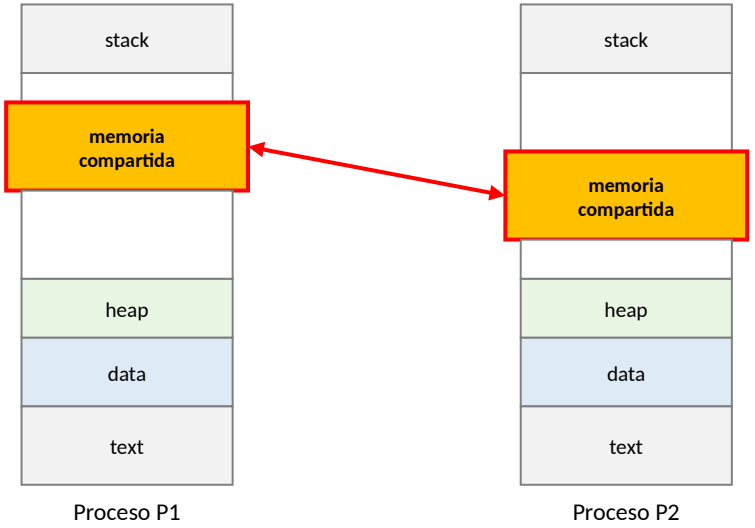
Contenido

1. Qué es la memoria compartida
2. Comparación con tuberías y colas de mensajes
3. Ciclo de vida de un segmento
4. Las llamadas shmget, shmat, shmdt y shmctl
5. Ejemplos: espera activa y sincronización con una señal
6. Resumen

Memoria compartida

- Un segmento de **memoria compartida** es una región que el kernel mapea en el espacio de direcciones de varios procesos.
- A diferencia de tuberías y colas, **no hay copia**: todos los procesos leen y escriben directamente sobre el mismo segmento.
- Es el mecanismo IPC más **rápido**, porque evita el paso por el kernel en cada operación.
- Como contrapartida, el acceso concurrente exige **sincronización explícita** (semáforos o señales): la memoria compartida no ordena por sí sola las lecturas y escrituras.
- Se identifica con un entero (`shmid`), localizado mediante una clave (`key_t`), igual que las colas de mensajes.

Dos procesos con un segmento compartido:



Ambos procesos ven las mismas páginas físicas, mapeadas entre heap y stack; la dirección virtual que devuelve shmat puede ser distinta en cada uno.

Comparación con tuberías y colas de mensajes

Mecanismo	Cómo se mueven los datos	Sincronización	Velocidad
Tubería	Copia usuario → kernel → usuario, en orden (flujo de bytes)	Implícita (read/write bloquean)	Media
Cola de mensajes	Copia usuario → kernel → usuario, por mensajes con tipo	Implícita (msgsnd/msgrcv bloquean)	Media
Memoria compartida	Sin copia: acceso directo de todos los procesos al mismo segmento	Explícita (semáforos o señales)	Alta

- La memoria compartida traslada al programador la responsabilidad que antes asumía el kernel: decidir **cuándo** es seguro leer o escribir.
- En los ejemplos de esta presentación se usan dos técnicas de sincronización: espera activa sobre un valor centinela, y una señal (`SIGUSR1`) que avisa cuando los datos están listos.

Llamadas al sistema para memoria compartida

1. Crear u obtener un segmento
   - Crear: tamaño dado en bytes **(shmget)**

   → Solo un proceso

2. Operaciones sobre el segmento
   - Adjuntar al espacio de direcciones **(shmat)**
   - Separar del espacio de direcciones **(shmdt)**

   → Todos los procesos

3. Control del segmento
   - Consultar, modificar o eliminar **(shmctl – IPC_RMID)**

   → Solo un proceso

Mismo esquema que las colas de mensajes: un proceso crea el recurso, todos lo usan y uno lo elimina.

Ciclo de vida de un segmento compartido

se repite mientras dure el uso (cualquier proceso que conozca el shmid)

1. Crear / obtener
`shmget()`

Un solo proceso lo crea; los demás obtienen el mismo shmid usando la misma clave.

2. Adjuntar
`shmat()`

Mapea el segmento en el espacio de direcciones del proceso y devuelve un puntero.

2. Separar
`shmdt()`

Retira el segmento del espacio de direcciones (no lo elimina del kernel).

3. Eliminar
`shmctl(IPC_RMID)`

Borra el segmento del kernel. Si nadie lo elimina, sobrevive a todos los procesos.

Mismo patrón que las colas de mensajes (System V IPC):

`msgget()`	`msgsnd()` / `msgrcv()`	`msgctl(IPC_RMID)`

En ambos casos: obtener un identificador a partir de una clave, operar sobre el recurso y, al terminar, eliminarlo del kernel.

Llamada al sistema ***shmget***

int shmget(key_t key, size_t size, int shmflg);

Crea un segmento de memoria compartida nuevo o devuelve el identificador del segmento asociado a la clave.

Include	`<sys/types.h>` `<sys/ipc.h>` `<sys/shm.h>`
Retorna	**Éxito:** identificador único del segmento (`shmid`). **Falla:** -1 y `errno` indica la causa.
Argumentos	• ***key_t key***: clave para crear o acceder al segmento (`ftok()` o `IPC_PRIVATE`). • ***size_t size***: tamaño en bytes a crear; `0` para acceder a un segmento existente. • ***int shmflg***: condición de creación y permisos de acceso (`IPC_CREAT`, `0666`).
Nota	El segmento queda en el kernel hasta que se elimina con `shmctl(IPC_RMID)` o `ipcrm`.

shmget(): valores de los argumentos y límites

Argumento	Valor	Significado
`key`	`IPC_PRIVATE`	Siempre crea un segmento nuevo; se comparte heredando el shmid (procesos emparentados, fork).
	`ftok(path, id)`	Clave pública compartida entre procesos independientes.
`size`	`N`	Número de bytes a reservar (se redondea al tamaño de página).
`shmflg`	`IPC_CREAT | 0666`	Crea el segmento si no existe; si existe, lo abre.

Límite (Linux)	Valor típico	Parámetro del kernel
Tamaño máximo de un segmento	prácticamente ilimitado en 64 bits (≈ 16 EiB)	`/proc/sys/kernel/shmmax`
Número máximo de segmentos	4 096	`/proc/sys/kernel/shmmni`
Páginas totales para segmentos compartidos	prácticamente ilimitado en 64 bits	`/proc/sys/kernel/shmall`

Valores por omisión en Linux de 64 bits (pueden variar): cat /proc/sys/kernel/shmmax shmmni shmall.

Llamada al sistema ***shmat***

void *shmat(int shmid, const void *shmaddr, int shmflg);

Adjunta (mapea) el segmento referenciado en el espacio de direcciones del proceso que llama.

Include	`<sys/shm.h>`
Retorna	**Éxito:** puntero al primer byte del segmento. **Falla:** `(void *) -1` (no `NULL`) y `errno` (`EINVAL`, `EACCES`).
Argumentos	• ***int shmid***: identificador válido del segmento. • ***const void *shmaddr***: dirección sugerida; con `NULL` (o `0`) el kernel elige la dirección. • ***int shmflg***: permisos y condiciones de adjunción (p. ej. `SHM_RDONLY`).
Nota	El `void *` se asigna a un puntero del tipo compartido (`int *p = shmat(id, NULL, 0);`) y se compara con `(void *) -1` para detectar el error.

Llamada al sistema ***shmdt***

int shmdt(const void *shmaddr);

Separa el segmento del espacio de direcciones del proceso que llama; no lo elimina del kernel.

Include	`<sys/shm.h>`
Retorna	**Éxito:** `0`. **Falla:** `-1` y `errno` (`EINVAL`).
Argumentos	• ***const void *shmaddr***: la dirección devuelta por `shmat` (el puntero a la memoria compartida).
Nota	El segmento sigue existiendo hasta que se elimina con `shmctl(IPC_RMID)`; `shmdt` solo lo desconecta de este proceso.

Llamada al sistema ***shmctl***

int shmctl(int shmid, int cmd, struct shmid_ds *buf);

Consulta, modifica o elimina un segmento y su estructura de control en el kernel.

Include	`<sys/shm.h>`
Retorna	**Éxito:** `0`. **Falla:** `-1` y `errno` (`EINVAL`, `EACCES`, `EIDRM`, `EPERM`).
Argumentos	• ***int shmid***: identificador válido del segmento. • ***int cmd***: operación a realizar (`IPC_STAT`, `IPC_SET`, `IPC_RMID`, `SHM_LOCK`, `SHM_UNLOCK` en Linux). • ***struct shmid_ds *buf***: estructura de control que se lee o escribe (`NULL` con `IPC_RMID`).

Operaciones de *shmctl()*

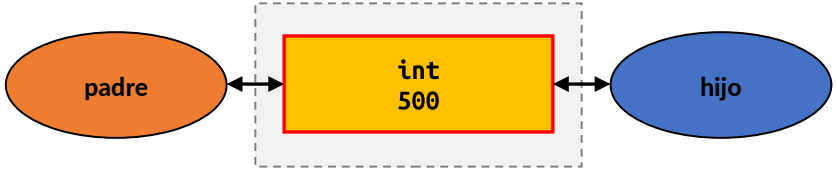
cmd	Efecto	Observaciones
IPC_STAT	Copia en buf la estructura shmid_ds del segmento.	Solo requiere permiso de lectura.
IPC_SET	Modifica algunos campos de shm_perm: propietario, grupo y permisos.	Solo el propietario, el creador o un proceso privilegiado.
IPC_RMID	Marca el segmento para eliminarlo; se destruye cuando el último proceso lo separa con shmdt.	El segmento no se libera de inmediato si sigue adjunto en algún proceso.
SHM_LOCK	Fija el segmento en memoria: no puede ir a swap (específico de Linux).	Requiere privilegios (CAP_IPC_LOCK) o caber en RLIMIT_MEMLOCK.
SHM_UNLOCK	Permite de nuevo que el segmento vaya a swap.	Mismos requisitos que SHM_LOCK.

A diferencia de `msgctl(IPC_RMID)`, que elimina la cola de inmediato, `shmctl(IPC_RMID)` solo marca el segmento para eliminarlo cuando el último proceso lo separe con `shmdt`.

Ejemplo 1: el padre escribe un entero, el hijo lo lee

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/stat.h>
int main(void){
  volatile int *ptr;   /* volatile: releer en cada vuelta */
  int shm_id = shmget(IPC_PRIVATE, sizeof(int),
                      IPC_CREAT | S_IRUSR | S_IWUSR);
  if(shm_id == -1){ perror("shmget"); exit(1); }
  ptr = shmat(shm_id, NULL, 0);
  if(ptr == (void *) -1){ perror("shmat"); exit(1); }
  if(!fork()){
    printf("[%d]child process started\n", getpid());
    do{}while(*ptr == 0);           /* espera activa */
    printf("[%d]shm_value->%d\n", getpid(), *ptr);
    shmdt((const void *) ptr);
  }
  else{
    printf("[%d]parent process started\n", getpid());
    sleep(2);
    *ptr = 500;
    printf("[%d]shm_value->%d\n", getpid(), *ptr);
    wait(NULL);
    shmdt((const void *) ptr);
    shmctl(shm_id, IPC_RMID, NULL);
  }
  return EXIT_SUCCESS;
}
```

```
[7712]parent process started
[7713]child process started
[7712]shm_value->500
[7713]shm_value->500
```



El hijo hace **espera activa** (`do{}while(*ptr==0)`) hasta que el padre escribe un valor distinto de 0. `volatile` obliga a releer la memoria en cada vuelta: sin él, con `-O2` el bucle no terminaría.

Ejemplo 2: mensajes de texto hasta "quit"

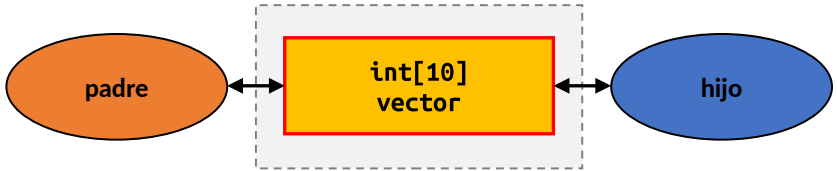
```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/stat.h>
#define SHM_SIZE 256
int main(void){
  char *ptr;
  int shm_id = shmget(IPC_PRIVATE, SHM_SIZE,
                      IPC_CREAT | S_IRUSR | S_IWUSR);
  if(shm_id == -1){ perror("shmget"); exit(1); }
  ptr = shmat(shm_id, NULL, 0);
  if(ptr == (void *) -1){ perror("shmat"); exit(1); }
  if(!fork()){
    printf("[%d]child process started\n", getpid());
    while(strcmp(ptr, "quit") != 0)  /* espera activa */
      usleep(1000);
    printf("[%d]shm_value->%s\n", getpid(), ptr);
    shmdt(ptr);
  }
  else{
    printf("[%d]parent process started\n", getpid());
    char msg[SHM_SIZE];
    do{
      if(fgets(msg, SHM_SIZE, stdin) == NULL)
        strcpy(msg, "quit");
      msg[strcspn(msg, "\n")] = '\0';
      strcpy(ptr, msg);
    }while(strcmp(ptr, "quit") != 0);
    wait(NULL);
    shmdt(ptr);
    shmctl(shm_id, IPC_RMID, NULL);
  }
  return EXIT_SUCCESS;
}
```



El hijo compara el contenido con `strcmp(ptr, "quit")` cada milisegundo: sigue siendo espera activa, ahora sobre una cadena. El padre sobrescribe el mismo buffer en cada mensaje: no hay cola, solo el último valor.

Ejemplo 3: sincronizar con una señal (SIGUSR1)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/stat.h>
void sig_handler(int s){ (void) s; }
int main(void){
  pid_t child;
  int *ptr;
  int shm_id = shmget(IPC_PRIVATE, sizeof(int) * 10,
                      IPC_CREAT | S_IRUSR | S_IWUSR);
  if(shm_id == -1){ perror("shmget"); exit(1); }
  ptr = shmat(shm_id, NULL, 0);
  if(ptr == (void *) -1){ perror("shmat"); exit(1); }
  signal(SIGUSR1, sig_handler);
  child = fork();
  if(!child){
    printf("[%d]child process started\n", getpid());
    pause();
    printf("[%d]reading shm_values:\n", getpid());
    for(int i = 0; i < 10; i++)
      printf("%d ", ptr[i]);
    printf("\n");
    shmdt(ptr);
  }
  else{
    printf("[%d]parent process started\n", getpid());
    int vect[10] = {10, 20, 5, 0, 2, 14, 1, 8, 9, 0};
    usleep(3000);
    for(int i = 0; i < 10; i++)
      ptr[i] = vect[i];
    printf("[%d]written shm_values.\n", getpid());
    kill(child, SIGUSR1);
    wait(NULL);
    shmdt(ptr);
    shmctl(shm_id, IPC_RMID, NULL);
  }
  return EXIT_SUCCESS;
}
```



El hijo se bloquea en `pause()` en vez de sondear: el padre escribe los diez enteros y luego `kill(child, SIGUSR1)` despierta al hijo. Memoria compartida para los datos, señal para la sincronización.

Resumen

- Un segmento compartido se mapea directamente en el espacio de direcciones de varios procesos: **sin copias**.
- `shmget` crea u obtiene el segmento; `shmat` lo adjunta y devuelve un puntero; `shmdt` lo separa; `shmctl` lo controla o elimina.
- No hay sincronización implícita: hay que coordinarse con espera activa (poco recomendable), semáforos o señales.
- `shmctl(IPC_RMID)` marca el segmento para borrarlo; desaparece cuando el último proceso lo separa con `shmdt`.
- Frente a tuberías y colas: más rápida, pero el programador asume la sincronización que antes daba el kernel.

Referencias y lecturas

- Kerrisk, M. *The Linux Programming Interface*, cap. 48: System V Shared Memory. No Starch Press, 2010.
- Stevens, W. R. *UNIX Network Programming*, *vol. 2: Interprocess Communications*, cap. 12. Prentice Hall, 1999.
- Carretero, J. et al. *Sistemas operativos: una visión aplicada*, servicios POSIX de comunicación. McGraw-Hill, 2007.
- Páginas de manual: `man 2 shmget`, `man 2 shmat`, `man 2 shmctl`, `man 7 sysvipc`.
- Código de los ejemplos: `example_codes/05-shm/` en el repositorio del curso.